

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



BUILD A SCROLLABLE LIST

Oleh:

Bi'ahlil Haq Aulia Akbar Awaludin

NIM. 2010817110011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2024**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN II
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Bi'ahlil Haq Aulia Akbar Awaludin
NIM : 2010817110011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Andreyan Rizky Baskara, S.Kom., M.Kom.
NIP. 19930703 201903 01 011

DAFTAR ISI

LEMBAR PENGESAHAN	1
DAFTAR ISI	2
DAFTAR GAMBAR	3
DAFTAR TABEL	4
SOAL	5
A. Source Code	8
Compose	8
XML	14
B. Output Program	23
C. Pembahasan	25
Jawaban Soal 1	25
Jawaban Soal 2	25
D. Tautan Git	25

DAFTAR GAMBAR

Gambar 1. Contoh UI List	6
Gambar 2. Contoh UI Detail	7
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose	23
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose	23
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML	24
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML	24

DAFTAR TABEL

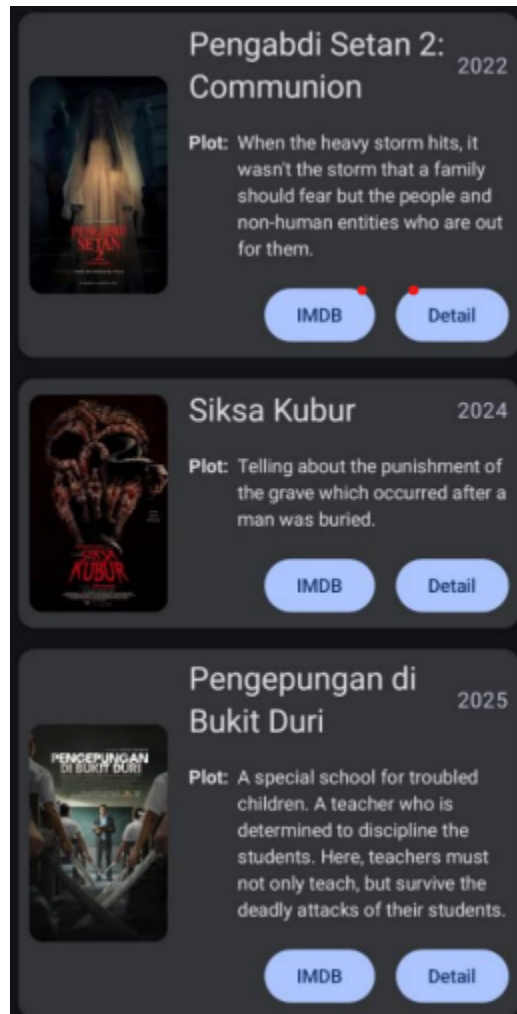
Tabel 1. Source Code Jawaban Soal 1	8
Tabel 2. Source Code Jawaban Soal 1	9
Tabel 3. Source Code Jawaban Soal 1	12
Tabel 4. Source Code Jawaban Soal 1	13
Tabel 5. Source Code Jawaban Soal 1	14
Tabel 6. Source Code Jawaban Soal 1	15
Tabel 7. Source Code Jawaban Soal 1	17
Tabel 8. Source Code Jawaban Soal 1	19
Tabel 9. Source Code Jawaban Soal 1	21

SOAL

Soal Praktikum:

1. Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:
 - a. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
 - b. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
 - c. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
 - d. Terdapat 2 button dalam list, dengan fungsi berikut:
 - i. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - ii. Button kedua menggunakan Navigation component untuk membuka laman detail item
 - e. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
 - f. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
 - g. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
 - h. Aplikasi berbasis XML harus menggunakan ViewBinding
2. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail

A. Source Code

Compose MovieListViewModel.kt

```
1 package com.example.movielist.ui.movielist
2
3 import androidx.compose.runtime.State
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.lifecycle.ViewModel
6 import androidx.lifecycle.viewModelScope
7 import com.example.movielist.data.Result
8 import com.example.movielist.data.repository.MovieRepository
9 import
10 com.example.movielist.data.repository.impl.FakeMovieRepository
11
12 import com.example.movielist.domain.model.Movie
13 import kotlinx.coroutines.flow.MutableSharedFlow
14 import kotlinx.coroutines.flow.asSharedFlow
15 import kotlinx.coroutines.launch
16 import timber.log.Timber
17
18
19 class MovieListViewModel(
20     val movieRepository : MovieRepository =
21     FakeMovieRepository()
22 ) : ViewModel() {
23
24     private val _eventFlow = MutableSharedFlow<UiEvent>()
25     val eventFlow = _eventFlow.asSharedFlow()
26
27     private val _movieListState =
28     mutableStateOf(MovieListState())
29     val movieListState: State<MovieListState> =
30     _movieListState
31
32     init {
33         getAllMovies()
34     }
35
36     fun getAllMovies() {
37         viewModelScope.launch {
38             _movieListState.value = MovieListState(isLoading
39 = true)
40
41             movieRepository.getAllMovies().collect { result
42 ->
43                 when (result) {
44                     is Result.Success<List<Movie>> -> {
45                         _movieListState.value =
46 MovieListState(movies = result.data, isLoading = false)
47
```

48	
49	Timber.tag("MovieListViewModel").i("Movies:
50	\${_movieListState.value}")
51	
52	}
53	is Result.Error -> {
54	_movieListState.value =
55	MovieListState(isLoading = false)
56	_eventFlow.emit(
57	UiEvent.ShowSnackbar(
58	message =
59	result.exception.message ?: "Unknown Error"
60	)
61	)
62	}
63	}
64	}
65	
66	
67	
68	fun onOpenUrlClicked(url: String) {
69	viewModelScope.launch {
70	if (url.isNotBlank()) {
71	_eventFlow.emit(UiEvent.OpenUrl(url))
72	Timber.tag("MovieListViewModel").i("intent
73	into URL: \$url")
74	} else {
75	_eventFlow.emit(UiEvent.ShowSnackbar("URL is
76	not valid"))
77	}
78	}
79	}
80	}
81	
82	data class MovieListState(
83	val movies: List<Movie> = emptyList(),
84	val isLoading: Boolean = true
85	)
86	
87	sealed class UiEvent {
88	data class ShowSnackbar(val message: String) : UiEvent()
89	data class OpenUrl(val url: String) : UiEvent()
90	}

Tabel 1. Source Code Jawaban Soal 1

MovieListScreen.kt

1	package com.example.movielist.ui.movielist
2	
3	import android.annotation.SuppressLint
4	import android.content.ActivityNotFoundException

```

5      import android.content.Intent
6      import androidx.compose.foundation.layout.Arrangement
7      import androidx.compose.foundation.layout.Box
8      import androidx.compose.foundation.layout.PaddingValues
9      import androidx.compose.foundation.layout.fillMaxSize
10     import androidx.compose.foundation.layout.padding
11     import androidx.compose.foundation.lazy.LazyColumn
12     import androidx.compose.foundation.lazy.items
13     import androidx.compose.material3.CircularProgressIndicator
14     import androidx.compose.material3.Scaffold
15     import androidx.compose.material3.SnackbarDuration
16     import androidx.compose.material3.SnackbarHost
17     import androidx.compose.material3.SnackbarHostState
18     import androidx.compose.material3.Surface
19     import androidx.compose.runtime.Composable
20     import androidx.compose.runtime.LaunchedEffect
21     import androidx.compose.runtime.remember
22     import androidx.compose.ui.Alignment
23     import androidx.compose.ui.Modifier
24     import androidx.compose.ui.platform.LocalContext
25     import androidx.compose.ui.res.stringResource
26     import androidx.compose.ui.tooling.preview.Preview
27     import androidx.compose.ui.unit.dp
28     import androidx.core.net.toUri
29     import com.example.movielist.domain.model.Movie
30     import com.example.movielist.ui.components.MovieCard
31     import com.example.movielist.ui.theme.MovieListTheme
32     import kotlinx.coroutines.flow.collectLatest
33     import timber.log.Timber
34
35     @Composable
36     fun MovieListScreen(
37         viewModel: MovieListViewModel,
38         detailOnClick: (Movie) -> Unit,
39         modifier: Modifier = Modifier
40     ) {
41         val state = viewModel.movieListState.value
42         val context = LocalContext.current
43         val snackbarHostState = remember { SnackbarHostState() }
44
45         LaunchedEffect(key1 = true) {
46             viewModel.eventFlow.collectLatest { event ->
47                 when (event) {
48                     is UiEvent.ShowSnackbar -> {
49                         snackbarHostState.showSnackbar(
50                             message = event.message,
51                             duration = SnackbarDuration.Short
52                         )
53                     }
54                     is UiEvent.OpenUrl -> {
55                         try {

```

```

57                                     val intent =
58 Intent(Intent.ACTION_VIEW, event.url.toUri())
59                                     context.startActivity(intent)
60                                     } catch (e: ActivityNotFoundException) {
61                                     Timber.tag("MovieListScreen")
62                                     .e("Error opening URL:
63 ${e.message} ${event.url}")
64                                     }
65                                     }
66                                     }
67                                     }
68     }
69     Scaffold(
70         snackbarHost = { SnackbarHost(snackbarHostState) },
71         modifier = modifier
72     ) { paddingValues ->
73         Box(
74             modifier = Modifier
75                 .fillMaxSize()
76                 .padding(paddingValues)
77         ) {
78             // Tampilkan loading
79             if (state.isLoading) {
80                 CircularProgressIndicator(modifier =
81 Modifier.align(Alignment.Center))
82             } else {
83                 LazyColumn(
84                     contentPadding = PaddingValues(16.dp),
85                     verticalArrangement =
86 Arrangement.spacedBy(16.dp)
87                 ) {
88                     items(state.movies) { movie ->
89                                     val imdbUrl =
90 stringResource(movie.imdbLink)
91                                     MovieCard(
92                     image = movie.image,
93                     titleText = movie.titleText,
94                     yearText = movie.yearText,
95                     descriptionText =
96 movie.descriptionText,
97                     openBrowserClick = {
98
99 viewModel.onOpenUrlClicked(imdbUrl)
100                                     },
101                                     detailOnClick = {
102                                     detailOnClick(movie)
103                                     }
104                                     )
105                                     Timber.tag("MovieProperty").d("Movie
106 Detail: $movie")
107                                     }
108                                     }

```

```

109         }
110     }
111 }
112 }
113
114 @SuppressLint("ViewModelConstructorInComposable")
115 @Preview
116 @Composable
117 private fun MovieListScreenPrev() {
118     MovieListTheme(darkTheme = true) {
119         Surface {
120             MovieListScreen(viewModel =
121 MovieListViewModel(), detailOnClick = {})
122         }
123     }
124 }
125 }

```

Tabel 2. Source Code Jawaban Soal 1

MovieDetailViewModel.kt

```

1 package com.example.movielist.ui.moviesdetail
2
3 import androidx.compose.runtime.State
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.lifecycle.ViewModel
6 import androidx.lifecycle.viewModelScope
7 import com.example.movielist.data.Result
8 import com.example.movielist.data.repository.MovieRepository
9 import
10 com.example.movielist.data.repository.impl.FakeMovieRepository
11
12 import com.example.movielist.domain.model.Movie
13 import com.example.movielist.ui.movielist.UiEvent
14 import kotlinx.coroutines.flow.MutableSharedFlow
15 import kotlinx.coroutines.flow.asSharedFlow
16 import kotlinx.coroutines.launch
17 import timber.log.Timber
18
19
20 class MovieDetailViewModel(
21     private val movieRepository: MovieRepository =
22 FakeMovieRepository(),
23 ) : ViewModel() {
24     private val _movieDetailState =
25 mutableStateOf(MovieDetailState())
26     val movieDetailState: State<MovieDetailState> =
27 _movieDetailState
28
29     private val _eventFlow =
30 MutableSharedFlow<UiEvent.ShowSnackbar>()
31     val eventFlow = _eventFlow.asSharedFlow()
32

```

33	fun getMovieDetail(id: Int?) {
34	if (id == null) {
35	viewModelScope.launch {
36	_eventFlow.emit(UiEvent.ShowSnackbar("Movie
37	ID tidak valid."))
38	}
39	return
40	}
41	viewModelScope.launch {
42	_movieDetailState.value =
43	MovieDetailState(isLoading = true)
44	
45	movieRepository.getMovie(id).collect { result ->
46	when (result) {
47	is Result.Success -> {
48	_movieDetailState.value =
49	MovieDetailState(
50	movie = result.data,
51	isLoading = false
52	)
53	
54	Timber.tag("MovieDetailViewModel").i("Get into Movie Detail:
55	\${_movieDetailState.value}")
56	}
57	
58	is Result.Error -> {
59	_movieDetailState.value =
60	MovieDetailState(isLoading = false)
61	_eventFlow.emit(
62	UiEvent.ShowSnackbar(
63	message =
64	result.exception.message ?: "Movie not found"
65	)
66	}
67	}
68	}
69	}
70	}
71	}
72	
73	
74	data class MovieDetailState(
75	val movie: Movie? = null,
76	val isLoading: Boolean = true
77	)

Tabel 3. Source Code Jawaban Soal 1

FakeRepository.kt

1	package com.example.movielist.data.repository.impl
2	
3	import com.example.movielist.data.Result

```

4      import com.example.movielist.data.repository.MovieRepository
5      import com.example.movielist.domain.model.Movie
6      import kotlinx.coroutines.delay
7      import kotlinx.coroutines.flow.Flow
8      import kotlinx.coroutines.flow.flow
9      import kotlinx.io.IOException
10
11     class FakeMovieRepository : MovieRepository {
12
13         private val movies = listOf(venom, kiki, dayLater,
14         turningRed, vhs, castle)
15
16         override fun getMovie(postId: Int?): Flow<Result<Movie>>
17     {
18             return flow {
19                 delay(1000)
20
21                 if (postId == null) {
22                     emit(Result.Error(Exception("Movie ID can't
23 be null")))
24                     return@flow // Hentikan eksekusi flow
25                 }
26
27                 val movieById: Movie? = movies.find { it.id ==
28 postId }
29
30                 if (movieById != null) {
31                     emit(Result.Success(movieById))
32                 } else {
33                     emit(Result.Error(Exception("Movie with id
34 $postId not found")))
35                 }
36             }
37         }
38
39         override suspend fun getAllMovies():
40 Flow<Result<List<Movie>>> {
41             return flow {
42                 try {
43                     delay(1000)
44                     emit(Result.Success(movies))
45                 } catch (e: IOException) {
46                     emit(Result.Error(Exception(e.message)))
47                 }
48             }
49         }
50     }

```

Tabel 4. Source Code Jawaban Soal 1

XML
MainActivity.kt


```

11
12 class MovieListAdapter(
13     private val onDetailClick: (MovieModel) -> Unit,
14     private val onBrowserClick: (MovieModel) -> Unit
15 ) : ListAdapter<MovieModel,
16     MovieListAdapter.MovieViewHolder>(MovieDiffCallback()) {
17
18     override fun onCreateViewHolder(parent: ViewGroup,
19     viewType: Int): MovieViewHolder {
20         val binding =
21     ItemMovieBinding.inflate(LayoutInflater.from(parent.context)
22     , parent, false)
23     return MovieViewHolder(binding)
24     }
25
26     override fun onBindViewHolder(holder: MovieViewHolder,
27     position: Int) {
28         val movie = getItem(position)
29         holder.bind(movie)
30     }
31
32     inner class MovieViewHolder(private val binding:
33     ItemMovieBinding) :
34     RecyclerView.ViewHolder(binding.root) {
35         fun bind(movie: MovieModel) {
36             val context = binding.root.context
37             val titleText = getString(context,
38     movie.titleText)
39             val yearText = getString(context,
40     movie.yearText)
41             val descriptionText = getString(context,
42     movie.descriptionText)
43
44
45     binding.ivMoviePoster.setImageResource(movie.image)
46             binding.tvMovieTitle.text = titleText
47             binding.tvMovieYear.text = yearText
48             binding.tvMovieDescription.text =
49     descriptionText
50             binding.btnItemDetail.setOnClickListener {
51                 onDetailClick(movie)
52             }
53
54             binding.btnItemBrowser.setOnClickListener {
55                 onBrowserClick(movie)
56             }
57         }
58     }
59 }
60
61 class MovieDiffCallback :
62     DiffUtil.ItemCallback<MovieModel>() {

```

63	override fun areItemsTheSame(oldItem: MovieModel,
64	newItem: MovieModel): Boolean {
65	return oldItem.id == newItem.id
66	}
67	
68	override fun areContentsTheSame(oldItem: MovieModel,
69	newItem: MovieModel): Boolean {
70	return oldItem == newItem
	}
	}

Tabel 6. Source Code Jawaban Soal 1

MovieListFragment.kt

1	package com.example.movieslistxml.ui.movieslist
2	
3	import android.content.Intent
4	import android.os.Bundle
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import android.widget.Toast
9	import androidx.core.net.toUri
10	import androidx.core.view.isVisible
11	import androidx.fragment.app.Fragment
12	import androidx.fragment.app.viewModels
13	import androidx.lifecycle.ViewModel
14	import androidx.lifecycle.ViewModelProvider
15	import androidx.navigation.fragment.findNavController
16	import com.example.movieslistxml.MovieListAdapter
17	import
18	com.example.movieslistxml.data.repository.MovieRepository
19	import
20	com.example.movieslistxml.data.repository.impl.FakeMovieRepository
21	
22	import
23	com.example.movieslistxml.databinding.FragmentMovieListBinding
24	
25	import timber.log.Timber
26	
27	class MovieListFragment : Fragment() {
28	
29	private var _binding: FragmentMovieListBinding? = null
30	private val binding get() = _binding!!
31	
32	private val viewModel: MovieListViewModel by viewModels
33	{
34	MovieListViewModelFactory(FakeMovieRepository())
35	}
36	
37	private lateinit var movieAdapter: MovieListAdapter
38	

```

39         override fun onCreateView(
40             inflater: LayoutInflater, container: ViewGroup?,
41             savedInstanceState: Bundle?
42         ): View {
43             _binding =
44             FragmentMovieListBinding.inflate(inflater, container, false)
45             Timber.tag("MovieListFragment").d("onCreateView:
46             Binding inflated.")
47             return binding.root
48         }
49
50         override fun onViewCreated(view: View,
51             savedInstanceState: Bundle?) {
52             super.onViewCreated(view, savedInstanceState)
53             Timber.tag("MovieListFragment").d("onViewCreated:
54             View is created.")
55
56             setupRecyclerView()
57             observeViewModel()
58         }
59
60         private fun setupRecyclerView() {
61             movieAdapter = MovieListAdapter(
62                 onDetailClick = { movie ->
63                     // Logika untuk tombol Detail
64                     Timber.d("Detail button clicked for:
65                     ${movie.titleText}")
66                     val action =
67                     MovieListFragmentDirections.actionMovieListFragmentToMovieDe
68                     tailFragment(movie.id)
69                     findNavController().navigate(action)
70                 },
71                 onBrowserClick = { movie ->
72                     Timber.d("Browser button clicked for:
73                     ${movie.titleText}")
74                     val imdbUrl = getString(movie.imdbLink)
75                     val intent = Intent(Intent.ACTION_VIEW,
76                     imdbUrl.toUri())
77                     try {
78                         startActivity(intent)
79                     } catch (e: Exception) {
80                         Toast.makeText(requireContext(),
81                         e.message, Toast.LENGTH_SHORT).show()
82                     }
83                 }
84             )
85             binding.rvMovies.adapter = movieAdapter
86
87             Timber.tag("MovieListFragment").d("setupRecyclerView:
88             RecyclerView and Adapter are set up.")
89         }
90

```

91	private fun observeViewModel() {
92	viewModel.movies.observe(viewLifecycleOwner) {
93	movies ->
94	if (movies.isNotEmpty()) {
95	Timber.tag("MovieListFragment").i("Observing
96	movies: \${movies.size} items submitted to adapter.")
97	movieAdapter.submitList(movies)
98	binding.rvMovies.isVisible =
99	movies.isNotEmpty()
100	}
101	}
102	viewModel.isLoading.observe(viewLifecycleOwner) {
103	isLoading ->
104	Timber.tag("MovieListFragment").d("Observing
105	isLoading: \$isLoading")
106	binding.progressBar.isVisible = isLoading
107	if (isLoading) {
108	binding.rvMovies.isVisible = false
109	Timber.tag("MovieListFragment").d("Making
110	rvMovies invisible")
111	}
112	}
113	}
114	Timber.tag("MovieListFragment").d("observeViewModel:
115	Observers are set.")
116	}
117	
118	override fun onDestroyView() {
119	super.onDestroyView()
120	_binding = null
121	Timber.tag("MovieListFragment").d("onDestroyView:
122	Binding is now null.")
123	}
124	}
125	}
126	
127	class MovieListViewModelFactory(private val repository:
128	MovieRepository) : ViewModelProvider.Factory {
129	override fun <T : ViewModel> create(modelClass:
130	Class<T>): T {
131	if
132	(modelClass.isAssignableFrom(MovieListViewModel::class.java)
133	) {
134	@Suppress("UNCHECKED_CAST")
135	return MovieListViewModel(repository) as T
136	}
137	throw IllegalArgumentException("Unknown ViewModel
	class")
	}
	}

Tabel 7. Source Code Jawaban Soal 1

MovieDetailFragment.kt

```

1 package com.example.movielistxml.ui.moviedetail
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5 import
6 com.example.movielistxml.data.repository.MovieRepository
7 import android.os.Bundle
8 import android.view.LayoutInflater
9 import android.view.View
10 import android.view.ViewGroup
11 import androidx.core.view.isVisible
12 import androidx.fragment.app.Fragment
13 import androidx.fragment.app.viewModels
14 import androidx.navigation.fragment.navArgs
15 import
16 com.example.movielistxml.data.repository.impl.FakeMovieRepository
17
18 import
19 com.example.movielistxml.databinding.FragmentMovieDetailBinding
20
21 import timber.log.Timber
22 import kotlin.getValue
23
24 class MovieDetailFragment : Fragment() {
25
26     private var _binding: FragmentMovieDetailBinding? = null
27     private val binding get() = _binding!!
28
29     private val viewModel: MovieDetailViewModel by
30 viewModels {
31         MovieDetailViewModelFactory(FakeMovieRepository())
32     }
33
34     private val args: MovieDetailFragmentArgs by navArgs()
35
36     override fun onCreateView(
37         inflater: LayoutInflater, container: ViewGroup?,
38         savedInstanceState: Bundle?
39     ): View {
40         _binding =
41         FragmentMovieDetailBinding.inflate(inflater, container,
42         false)
43         Timber.tag("MovieDetailFragment").d("onCreateView:
44         Binding inflated.")
45         return binding.root
46     }
47
48     override fun onViewCreated(view: View,
49 savedInstanceState: Bundle?) {
50         super.onViewCreated(view, savedInstanceState)
51         Timber.tag("MovieDetailFragment").d("onViewCreated:
52         View is created.")

```

```

53         viewModel.getMovieDetail(args.movieId)
54         observeViewModel()
55     }
56
57     private fun observeViewModel() {
58         viewModel.movie.observe(viewLifecycleOwner) { movie
59 ->
60             movie?.let {
61
62                 Timber.tag("MovieDetailFragment").i("Observing      movies:
63 $movie items submitted to adapter.")
64
65                 binding.ivDetailPoster.setImageResource(it.image)
66                                     binding.tvDetailTitle.text =
67                 getString(it.titleText)
68                                     binding.tvDetailYear.text =
69                 getString(it.yearText)
70                                     binding.tvDetailDescription.text =
71                 getString(it.descriptionText)
72             }
73         }
74         viewModel.isLoading.observe(viewLifecycleOwner) {
75             isLoading ->
76                 Timber.tag("MovieListFragment").d("Observing
77 isLoading: $isLoading")
78                 binding.detailProgressBar.isVisible = isLoading
79
80                 binding.contentScrollView.isVisible = !isLoading
81                 Timber.tag("MovieListFragment").d("Making
82 rvMovies invisible")
83             }
84         }
85
86         override fun onDestroyView() {
87             super.onDestroyView()
88             _binding = null
89         }
90     }
91
92     class MovieDetailViewModelFactory(private val repository:
93 MovieRepository) : ViewModelProvider.Factory {
94         override fun <T : ViewModel> create(modelClass:
95 Class<T>): T {
96
97             if
98             (modelClass.isAssignableFrom(MovieDetailViewModel::class.jav
99 a)) {
100                 @Suppress("UNCHECKED_CAST")
101                 return MovieDetailViewModel(repository) as T
102             }
103             throw IllegalArgumentException("Unknown ViewModel
104 class")
105         }

```

105	}
106	

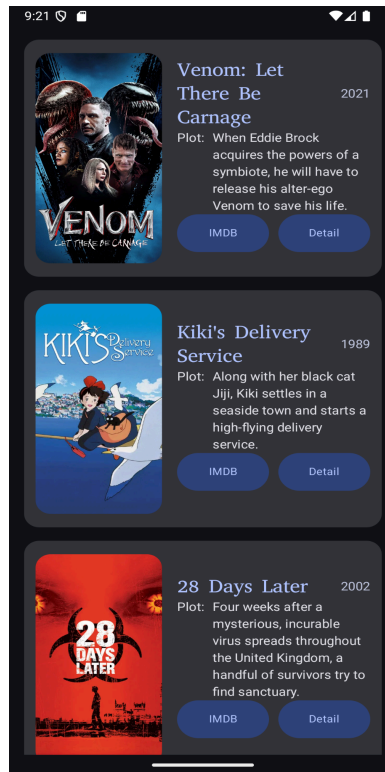
Tabel 8. Source Code Jawaban Soal 1

nav_graph.xml

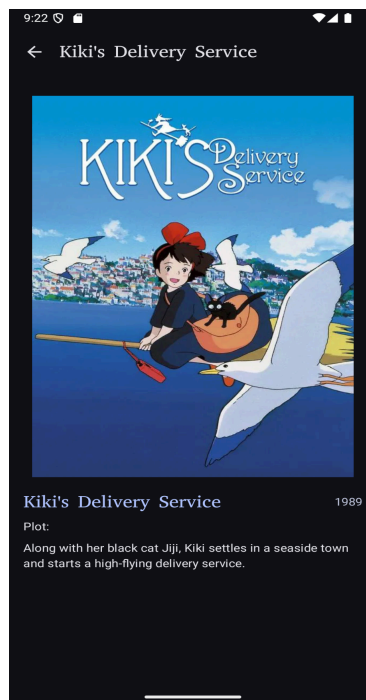
1	<?xml version="1.0" encoding="utf-8"?>
2	
3	<navigation
4	xmlns:android="http://schemas.android.com/apk/res/android"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	xmlns:tools="http://schemas.android.com/tools"
7	android:id="@+id/nav_graph"
8	app:startDestination="@id/movieListFragment">
9	
10	<fragment
11	android:id="@+id/movieListFragment"
12	
13	android:name="com.example.movieslistxml.ui.movieslist.MovieLis
14	tFragment">
15	<action
16	
17	android:id="@+id/action_movieListFragment_to_movieDetailFrag
18	ment"
19	app:destination="@id/movieDetailFragment" />
20	</fragment>
21	
22	<fragment
23	android:id="@+id/movieDetailFragment"
24	
25	android:name="com.example.movieslistxml.ui.moviesdetail.MovieD
26	etailFragment"
27	tools:layout="@layout/fragment_movie_detail">
28	<argument
29	android:name="movieId"
30	app:argType="integer" />
31	</argument>
32	
33	</navigation>

Tabel 9. Source Code Jawaban Soal 1

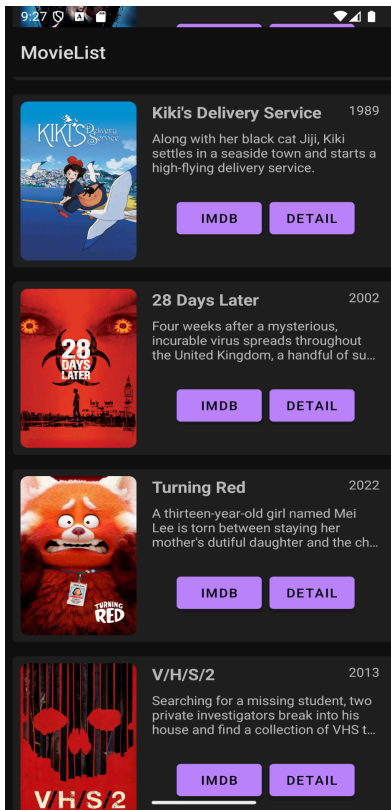
B. Output Program



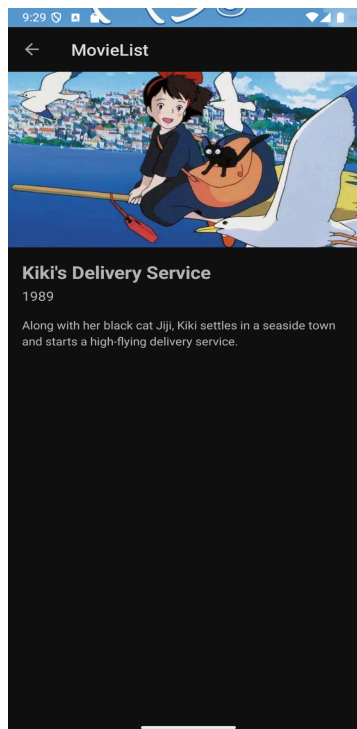
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 5. Screenshot Hasil Jawaban Soal 1 XML



Gambar 6. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

Jawaban Soal 1

compose

Pada File **MovieListViewModel** Line 68 - 80, itu adalah logik dari intent yang kemudian akan dipakai pada screen atau fragment. (1 D)

Pada File **MovieListScreen** Line 54 - 68, itu adalah penggunaan intent yang akan dijalankan oleh corotine jika fungsi viewmodel dipanggil.(1 D)

Pada File **MovieListScreen** Line 83 - 110, itu adalah penggunaan Lazycolum.(1 A)

XML

Pada File **MovieListAdapter** Line 26 - 56, itu adalah penggunaan penerapan viewbinding dari RecyclerView dimana loop viewbind untuk menampilkan ui setiap item.

Pada File **MovieListFragment** Line 58 - 86, itu adalah penerapan dari intent untuk keluar ke browser dan untuk navigasi ke detail fragment.

Pada File **nav_graph**, itu adalah penggunaan navigasi pada xml dan terdapat 2 fragment yang menjadi destinasi.

Jawaban Soal 2

Ada beberapa kelebihan recycle view dari pada compose salah satunya adalah: fleksibilitas dan kontrol sangat tinggi, performa sangat efisien dan spesifik. Berbeda compose terabstraksi, dan general efisien. Situasi dimana recyle view masih diperlukan: bekerja di proyek lama (legacy codebase), kebutuhan animasi item yang sangat kompleks, dan kebutuhan performa yang sangat spesifik.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/biahlil/Pemrograman-Mobile.git>